ReconForge Operator Runbooks

Practical end-to-end workflows for common assessment scenarios. Each runbook provides exact CLI commands, expected output files, findings interpretation, and manual follow-up guidance.

> **Important:** All commands assume you are in the ReconForge project root directory (`/path/to/reconforge/`). All external tools (nmap, gobuster, etc.) must be installed and on PATH. Run `--dry-run` first to verify commands before live execution.

Runbook 1: External Web Application Assessment

When to Use

- External web application penetration test
- No credentials or internal network access
- Target is a URL (e.g., `https://app.target.com`)
- Goal: map the attack surface, discover content, identify vulnerabilities

Prerequisites

- **Authorization:** Written scope/ROE covering the target URL
- **Tools required:** nmap, whatweb, wafw00f, gobuster or ffuf, nuclei, nikto (optional: wpscan, sqlmap)
- **Network:** Direct internet access to the target
- **Time estimate:** 20–60 minutes depending on target size and OPSEC mode

Step 1: Attack Surface Mapping

Map open ports and services on the target host to understand the full exposure before focusing on the web application.

```
python reconforge.py surface --target app.target.com --opsec normal -v
```

What this does:
- Phase `port_discovery` : SYN scan of top 1000 ports via nmap
- Phase `service_fingerprint` : Version detection on open ports
- Phase `vector_correlation` : Maps services to known attack vectors
- Phase `prioritization` : Scores and ranks the attack surface

Expected output directory: `outputs/app.target.com/surface/`

File	Contents
`raw/`	Raw nmap and tool output files
`parsed/`	Structured parsed results
`findings.json`	JSON findings array
`findings.md`	Markdown findings report
`session.md`	Timestamped session notes
`commands.log`	All commands executed

Key findings to look for:
- Open ports beyond 80/443 (admin interfaces, debug ports)
- Service versions with known CVEs
- HTTP services on non-standard ports (8080, 8443, etc.)

Step 2: Web Reconnaissance

Run the web module against the target URL. In normal OPSEC mode, this runs surface discovery, content enumeration, and vulnerability scanning (phases 1–3). The exploit phase (phase 4) is opt-in only.

```
python reconforge.py web --target https://app.target.com --opsec normal -v
```

To run with specific phases:

```
python reconforge.py web --target https://app.target.com --phases
surface,content,vuln -v
```

To include file extension fuzzing:

```
python reconforge.py web --target https://app.target.com --phases surface,content -e
php,asp,aspx -v
```

What each phase does:

Phase	Tools Used	Detection Noise
`surface`	whatweb, wafw00f, curl (headers)	Low
`content`	gobuster/ffuf (directory brute-force)	Medium
`vuln`	nikto, nuclei (CVE templates)	High
`exploit`	wpscan, sqlmap (opt-in, aggressive only)	Very High

Expected output directory: `outputs/https___app.target.com/web/`

File	Contents
`raw/`	Raw tool output (whatweb, wafw00f, gobuster, nikto, nuclei)
`parsed/`	Structured parsed results per tool
`findings.json`	All findings with severity, confidence, evidence
`findings.md`	Human-readable findings report
`loot.json`	Discovered credentials, tokens, interesting files
`session.md`	Session timeline
`commands.log`	Full command log

Key findings to look for:
- WAF detection results (impacts subsequent testing strategy)
- Technology stack (CMS, framework, language, server)
- Discovered directories and files (admin panels, backup files, config files)
- Known CVEs from nuclei templates
- Security header issues (missing HSTS, CSP, X-Frame-Options)

Step 3: API Assessment (if applicable)

If the surface or web phases reveal API endpoints (e.g., `/api/`, `/v1/`, Swagger/OpenAPI specs):

```
python reconforge.py api --target https://app.target.com/api/v1 --opsec normal -v
```

Expected output directory: `outputs/https___app.target.com_api_v1/api/`

Output structure mirrors the web module: `raw/`, `parsed/`, `findings.json`, `findings.md`, `loot.json`, `session.md`, `commands.log`.

Step 4: Stealth Alternative

If you need to minimize detection (e.g., testing IDS/WAF response or red team engagement):

```
python reconforge.py surface --target app.target.com --opsec stealth -v
python reconforge.py web --target https://app.target.com --opsec stealth -v
```

In stealth mode:
- Surface: Only top 100 ports, T1 timing, no version detection
- Web: Only the `surface` phase runs (whatweb + wafw00f — both low noise)
- Content enumeration, vulnerability scanning, and exploit phases are **blocked** by OPSEC policy

Findings Interpretation

Review `findings.md` in each module output directory. Findings are classified by:

- **Severity:** critical, high, medium, low, info
- **Confidence:** confirmed, high, medium, low, heuristic

Severity clamping rules apply: Heuristic-confidence findings are capped at `low` severity. The findings report notes when clamping occurs with a `[severity clamped: X→Y]` prefix.

Common Issues

Issue	Cause	Fix
Empty content enumeration results	WAF blocking, wrong wordlist	Try `--wordlist /path/to/custom.txt`, check wafw00f output
All phases skipped	OPSEC mode too restrictive	Use `--opsec normal` or `--opsec aggressive`
Tool not found errors	Missing external tool	Install per `tools.yaml` `install_cmd` entries
Timeouts on large scans	Default 600s too short	Use `--timeout 1200`

Manual Follow-Up Tasks

1. **Validate heuristic findings** — Any finding with `heuristic` confidence requires manual verification
2. **Test business logic** — ReconForge does not test business logic flaws (auth bypass, IDOR, race conditions)
3. **Manual exploitation** — Use identified CVEs and misconfigurations as starting points for manual exploitation
4. **Check for WAF bypass** — If WAF detected, attempt bypass techniques manually
5. **Credential testing** — If default/common credentials found in loot, validate manually

6. **SSL/TLS deep dive** — Run testssl.sh manually for comprehensive TLS assessment

---

Runbook 2: Internal AD-Focused Assessment

When to Use

- Internal network penetration test
- Active Directory environment
- You have network access to a Domain Controller
- Goal: enumerate AD, find attack paths, identify privilege escalation opportunities

Prerequisites

- **Authorization:** Written scope covering internal network and AD domain
- **Tools required:** nmap, enum4linux or enum4linux-ng, ldapsearch, smbclient, impacket suite (optional: bloodhound-python, netexec/crackmapexec)
- **Network:** Access to target DC (ports 88, 135, 139, 389, 445, 636 at minimum)
- **Credentials (optional but recommended):** Domain user credentials for authenticated enumeration
- **Time estimate:** 30–90 minutes depending on domain size and OPSEC mode

Step 1: Network Reconnaissance

Start with network recon to discover live hosts and services. This feeds into the AD module's conditional execution.

```
python reconforge.py network --target 10.10.10.0/24 --opsec normal -v
```

For a single DC target:

```
python reconforge.py network --target 10.10.10.1 --opsec normal -v
```

What each phase does:

Phase	Description
`discovery`	Host discovery via nmap ping sweep
`scanning`	Port scanning and service detection
`enumeration`	SMB/LDAP/NetBIOS deep enumeration (enum4linux, smbclient, ldapsearch)
`authentication`	Anonymous access checks; hydra brute-force only with `--brute-force`

Expected output directory: `outputs/10.10.10.1/network/` (or `outputs/10.10.10.0_24/network/` for CIDR)

File	Contents
`raw/`	Raw nmap, enum4linux, smbclient, ldapsearch output
`parsed/`	Structured parsed results
`findings.json`	Findings with severity and evidence
`findings.md`	Human-readable findings
`loot.json`	Discovered users, shares, credentials, services
`session.md`	Session notes timeline
`commands.log`	Command history

Step 2: Active Directory Enumeration

Run the AD module against the Domain Controller. Unauthenticated:

```
python reconforge.py ad --target 10.10.10.1 --domain corp.local --opsec normal -v
```

Authenticated (recommended for deeper enumeration):

```
python reconforge.py ad --target 10.10.10.1 --domain corp.local -u jsmith -p
'P@ssw0rd' --dc-ip 10.10.10.1 -v
```

To run specific phases only:

```
python reconforge.py ad --target 10.10.10.1 --domain corp.local --phases pass-
ive,identity -v
```

AD Module Phases:

Phase	Tools Used	What It Finds
`passive`	nmap (AD ports), ldapsearch (anonymous), smbclient (null session), DNS SRV	Domain info, anonymous access, AD services
`identity`	enum4linux-ng, impacket (GetADUsers, GetNPUsers, GetUserSPNs), ldapsearch	Users, groups, SPNs, AS-REP roastable accounts, Kerberoastable accounts
`configuration`	ldapsearch, nmap (SMB scripts), smbclient, impacket (rpcdump)	Password policies, trusts, GPOs, shares, DC configuration
`delegation`	ldapsearch, impacket (findDelegation)	Unconstrained/constrained/RBCD delegation configurations
`bloodhound`	bloodhound-python, netexec	AD graph data for BloodHound analysis (aggressive mode only)

Expected output directory: `outputs/10.10.10.1/ad/`

File	Contents
`raw/`	Raw tool output files
`parsed/`	Parsed structured data
`findings.json`	All AD findings
`findings.md`	Markdown findings report
`loot.json`	Users, hashes, credentials, shares, services
`attack_paths.md`	Identified attack paths with steps
`ad_summary.md`	AD environment summary
`quick_report.md`	Executive-level quick report
`session.md`	Session timeline
`commands.log`	All commands executed

Step 3: Aggressive Mode (Lab/CTF)

For full coverage in a lab environment:

```
python reconforge.py ad --target 10.10.10.1 --domain corp.local -u jsmith -p
'P@ssw0rd' --opsec aggressive -v
```

This enables all phases including `bloodhound` collection, RID cycling, full NSE scripts, and all impacket tools.

Step 4: Workflow Orchestration (Combined)

Chain network and AD modules automatically with the workflow engine:

```
python reconforge.py workflow --target 10.10.10.1 --modules network,ad --opsec normal
\
    --engagement "Q1 Internal Pentest" --client "Acme Corp" -v
```

The workflow engine:
- Runs network module first to discover services
- Automatically populates the workflow context with discovered ports/services
- Conditionally runs the AD module if AD-related services are found (LDAP, Kerberos, SMB)
- Passes credential data between modules via the CredentialVault

Workflow output: `outputs/workflow/workflow_YYYYMMDD_HHMMSS.json` and `outputs/workflow/engagement_YYYYMMDD_HHMMSS.json`

Key Findings Interpretation

Attack paths to prioritize:
- **AS-REP Roasting:** Accounts without Kerberos pre-authentication — crack offline
- **Kerberoasting:** Service accounts with SPNs — request TGS tickets and crack offline
- **Unconstrained Delegation:** Machines that can impersonate any user to any service
- **Constrained Delegation:** Machines allowed to delegate to specific services — potential for S4U abuse
- **RBCD (Resource-Based Constrained Delegation):** If MachineAccountQuota > 0, potential computer account creation
- **Null Session / Anonymous Access:** SMB shares or LDAP readable without credentials
- **Weak Password Policy:** Short minimum length, no complexity, no lockout

Loot to examine:
- `loot.json` → check for enumerated users (spray targets), hashes (crack with hashcat), discovered shares
- `attack_paths.md` → prioritized attack chains with step-by-step exploitation guidance

Common Issues

Issue	Cause	Fix
LDAP anonymous bind fails	DC configured to reject anonymous	Provide credentials with `-u` / `-p`
enum4linux-ng not found	Not installed	`pip install enum4linux-ng`
Impacket tools not found	Impacket not installed	`pip install impacket`
Bloodhound phase skipped	Not aggressive mode, or tool missing	Use `--opsec aggressive`, install bloodhound-python
Empty identity enumeration	Anonymous bind rejected, no creds	Provide domain credentials

Manual Follow-Up Tasks

1. **Crack hashes** — Feed NTLM hashes and Kerberos tickets from loot to hashcat/john
2. **Validate attack paths** — Use identified paths for manual exploitation (e.g., Rubeus, impacket-secretsdump)
3. **Lateral movement** — Use discovered credentials for Pass-the-Hash, Pass-the-Ticket
4. **BloodHound analysis** — Import collected data into BloodHound GUI for graph-based path analysis
5. **GPO abuse** — If writable GPOs found, test for GPO-based code execution
6. **Share analysis** — Manually examine accessible shares for sensitive data (scripts, configs, passwords)
7. **Password spraying** — Use enumerated usernames with common passwords (only if authorized)

Runbook 3: Authenticated API Assessment

When to Use

- API security assessment with provided credentials/tokens
- Testing REST/GraphQL/SOAP APIs
- You have a Bearer token, API key, or JWT
- Goal: discover endpoints, test authentication, fuzz parameters, check authorization controls

Prerequisites

- **Authorization:** Written scope covering the API endpoint(s)
- **Tools required:** httpx, ffuf, arjun, nuclei (all optional but recommended)
- **Credentials:** Bearer token, JWT, or API key
- **API documentation (optional):** OpenAPI/Swagger spec URL if available
- **Time estimate:** 15–45 minutes depending on API size

Step 1: API Discovery and Authentication Testing

Run the API module with your auth token:

```
python reconforge.py api --target https://api.target.com/v1 \
    --auth-token "Bearer eyJhbGciOiJIUzI1NiIs..." \
    --opsec normal -v
```

With custom headers (e.g., API key):

```
python reconforge.py api --target https://api.target.com/v1 \
    --header "X-Api-Key: abc123def456" \
    --opsec normal -v
```

With multiple headers:

```
python reconforge.py api --target https://api.target.com/v1 \
    --header "Authorization: Bearer eyJ..." \
    --header "X-Custom-Header: value" \
    --opsec normal -v
```

Step 2: Run Specific Phases

Discovery only (lowest noise):

```
python reconforge.py api --target https://api.target.com/v1 \
    --auth-token "Bearer eyJ..." \
    --phases discovery -v
```

Discovery + authentication testing:

```
python reconforge.py api --target https://api.target.com/v1 \
    --auth-token "Bearer eyJ..." \
    --phases discovery,authentication -v
```

Full assessment including authorization testing (opt-in):

```
python reconforge.py api --target https://api.target.com/v1 \
    --auth-token "Bearer eyJ..." \
    --phases discovery,authentication,fuzzing,authorization \
    --opsec aggressive -v
```

API Module Phases:

Phase	Tools Used	What It Tests
`discovery`	httpx, ffuf (endpoint enumeration), OpenAPI spec detection	API endpoints, technology stack, spec files
`authentication`	nuclei (auth templates), JWT analysis	Auth mechanisms, JWT weaknesses, token handling
`fuzzing`	ffuf (param fuzzing), arjun (param discovery)	Hidden parameters, input validation, injection points
`authorization`	Pattern-based BOLA/IDOR testing (opt-in)	Access control, privilege escalation, BOLA/IDOR

Expected output directory: `outputs/https___api.target.com_v1/api/`

File	Contents
`raw/`	Raw httpx, ffuf, arjun, nuclei output
`parsed/`	Structured parsed results per tool
`findings.json`	All findings with severity/confidence/evidence
`findings.md`	Markdown findings report
`loot.json`	Discovered tokens, endpoints, parameters
`session.md`	Session timeline
`commands.log`	Command history

Key Findings Interpretation

JWT issues to look for:
- Algorithm confusion (none, HS256 vs RS256)
- Weak signing secrets
- Missing expiration (`exp` claim)
- Sensitive data in payload (PII, credentials)
- Token not invalidated on logout

> **Note:** JWT analysis includes heuristic checks. Findings with `heuristic` confidence are capped at `low` severity and require manual validation.

Authorization flaws:
- BOLA/IDOR: Accessing other users' resources by changing IDs
- Missing function-level access control
- Privilege escalation via parameter manipulation
- Mass assignment vulnerabilities

> **Note:** Authorization fuzzing is pattern-based and may produce false positives. Always validate manually.

API-specific findings:
- Exposed Swagger/OpenAPI specs (information disclosure)
- Verbose error messages revealing stack traces
- Missing rate limiting
- Insecure CORS configuration
- API versioning issues

Step 3: Stealth API Recon

For minimal footprint (e.g., bug bounty with restrictive scope):

```
python reconforge.py api --target https://api.target.com/v1 \
    --auth-token "Bearer eyJ..." \
    --opsec stealth -v
```

In stealth mode, only the `discovery` phase runs with httpx probing and spec detection — no fuzzing, no auth testing, no authorization testing.

Common Issues

Issue	Cause	Fix
401/403 on all requests	Token expired or invalid	Refresh your token, check `--auth-token` format
Empty endpoint discovery	No wordlist match, or API uses UUIDs	Provide custom wordlist with `--wordlist`, try OpenAPI spec
Authorization phase skipped	Not in phases list or OPSEC blocks it	Add `authorization` to `--phases`, use `--opsec aggressive`
Rate limiting triggered	Too many requests	Use `--opsec stealth` or lower threads
ffuf/arjun not found	Tools not installed	Install per `tools.yaml` instructions

Manual Follow-Up Tasks

1. **Validate JWT findings** — Use jwt.io or jwt_tool to confirm algorithm confusion, weak secrets
2. **Test BOLA/IDOR manually** — Replace resource IDs with other users' IDs in authenticated requests
3. **Business logic testing** — Test workflow bypass, order manipulation, payment logic
4. **Rate limit bypass** — Test header-based bypasses (X-Forwarded-For, X-Real-IP)
5. **GraphQL introspection** — If GraphQL detected, run introspection queries manually

6. **Mass assignment** — Send extra fields in POST/PUT requests to test for unintended attribute updates
7. **Credential rotation** — Test if old tokens are properly invalidated after password change

---

General Notes

Dry Run Mode

Always preview commands before live execution on sensitive targets:

```
python reconforge.py web --target https://app.target.com --opsec normal --dry-run -v
```

Dry run logs all commands that **would** execute without actually running them.

Encrypted Loot

To encrypt loot files (credentials, hashes, tokens):

```
python reconforge.py network --target 10.10.10.1 --encrypt-loot
```

Loot is encrypted with Fernet symmetric encryption. The key is stored at `~/.reconforge/loot.key`. To decrypt:

```
from core.loot_manager import LootManager
plaintext = LootManager.load_encrypted("outputs/10.10.10.1/network/loot.json.enc")
```

Resuming Engagements

Save and resume workflow engagements:

```
# Start an engagement
python reconforge.py workflow --target 10.10.10.1 --engagement "Q1 Pentest" --client "
Acme" -v

# Resume from saved state
python reconforge.py workflow --target 10.10.10.1 --resume outputs/workflow/engage-
ment_20250321_143000.json
```

Output Directory Convention

All output follows the pattern:

```
outputs/<sanitized_target>/<module>/
├── raw/          # Unprocessed tool output
├── parsed/       # Structured parsed data
├── findings.json # Machine-readable findings
├── findings.md   # Human-readable findings
├── loot.json     # Extracted credentials, tokens, shares
├── session.md    # Timestamped session notes
└── commands.log  # All commands executed
```

AD module additionally produces: `attack_paths.md`, `ad_summary.md`, `quick_report.md`.

Workflow mode produces: `outputs/workflow/workflow_YYYYMMDD_HHMMSS.json`, `engagement_YYYYMMDD_HHMMSS.json`, `vault_YYYYMMDD_HHMMSS.json`.